

Spiking Neural Networks on Conventional Hardware: Accuracy Parity, Speed Disparity

Turing
RTSS Board
turingrtss@gmail.com

September 2026

Abstract

We implement leaky integrate-and-fire spiking neural networks with surrogate gradient training and compare them against conventional architectures (MLP, CNN, Transformer) on MNIST classification. Spiking convolutional networks achieve 97.41% accuracy, within 0.35 percentage points of conventional CNN (97.76%), demonstrating that the accuracy gap between spiking and conventional architectures is negligible on this task. However, on conventional CPU hardware, spiking networks are $36\times$ slower at inference and $30\times$ slower to train than equivalent conventional networks due to the sequential timestep computation that cannot exploit spike sparsity without dedicated neuromorphic hardware. We report accuracy, training time, inference latency, parameter counts, and spike statistics for eight model variants, providing a quantitative reference for the practical tradeoffs of neuromorphic computing on conventional processors.

1 Introduction

Spiking neural networks (SNNs) process information as discrete binary events (spikes) rather than continuous activations, mimicking the temporal dynamics of biological neurons [1]. On neuromorphic hardware such as Intel Loihi [2] and IBM TrueNorth [3], this spike-based computation translates to $100\text{--}1000\times$ energy savings over conventional GPU inference, because the hardware only consumes energy when a neuron fires.

The practical question for edge deployment is whether SNNs offer any advantage on conventional processors (CPUs, GPUs) that lack dedicated spike-routing circuitry. If SNNs can match conventional architectures on accuracy while maintaining parameter efficiency, they become attractive for memory-constrained devices even without the energy benefit.

This paper measures both sides: accuracy parity and speed disparity between spiking and conventional architectures on identical tasks and hardware.

2 Methods

2.1 Spiking Neuron Model

We use leaky integrate-and-fire (LIF) neurons with the following dynamics per timestep t :

$$U_t = \beta \cdot U_{t-1} + W \cdot X_t \tag{1}$$

$$S_t = \Theta(U_t - V_{\text{thr}}) \tag{2}$$

$$U_t \leftarrow U_t \cdot (1 - S_t) \tag{3}$$

where U_t is the membrane potential, $\beta = 0.9$ is the decay constant, W is the weight matrix, X_t is the input, $V_{\text{thr}} = 1.0$ is the firing threshold, $S_t \in \{0, 1\}$ is the spike output, and the membrane resets to zero after firing.

Input encoding uses rate coding: each pixel’s intensity becomes the probability of an input spike at each timestep.

2.2 Surrogate Gradient

The Heaviside step function Θ has zero gradient almost everywhere. We use a fast sigmoid surrogate [4] for backpropagation:

$$\frac{\partial S}{\partial U} \approx \frac{1}{(\alpha|U - V_{\text{thr}}| + 1)^2}, \quad \alpha = 25 \quad (4)$$

2.3 Architectures

Table 1: Model configurations.

Model	Type	Params	Timesteps
SNN-FC T=10/25/50	Spiking FC (2 LIF layers, 128 hidden)	101,770	10/25/50
SNN-Conv T=10/25	Spiking Conv (2 LIF conv + FC)	11,274	10/25
MLP	Conventional (2 hidden, 64 units)	55,050	–
CNN (LeNet)	Conventional (2 conv + FC)	54,314	–
Transformer	Conventional (2 layers, 4 heads)	19,242	–

2.4 Training

All models trained for 15 epochs with Adam ($\eta = 10^{-3}$) on a 20,000-sample MNIST subset, evaluated on the full 10,000-sample test set. Hardware: 2-core ARM64 CPU, 12 GB RAM, no GPU.

3 Results

Table 2: Full comparison: spiking vs. conventional architectures.

Model	Params	Acc.	Train (s)	Infer (ms)	Spikes
CNN	54,314	0.9776	154	0.10	–
SNN-Conv T=25	11,274	0.9741	4,663	3.77	26,707
Transformer	19,242	0.9699	159	0.12	–
SNN-Conv T=10	11,274	0.9693	3,227	1.57	9,596
MLP	55,050	0.9520	7	0.004	–
SNN-FC T=50	101,770	0.9488	898	1.63	1,980
SNN-FC T=25	101,770	0.9461	483	0.75	991
SNN-FC T=10	101,770	0.9419	202	0.30	409

3.1 Accuracy Parity

SNN-Conv T=25 achieves 97.41% accuracy, 0.35 percentage points below CNN (97.76%). SNN-FC T=25 achieves 94.61%, 0.59 pp below MLP (95.20%). In both cases, the spiking variant is within 1% of its conventional counterpart, demonstrating that surrogate gradient training closes the accuracy gap on this benchmark.

3.2 Speed Disparity

The inference time ratio reveals the computational cost of temporal simulation on conventional hardware:

- SNN-Conv vs CNN: $3.77/0.10 = 37.7\times$ slower
- SNN-FC vs MLP: $0.75/0.004 = 187.5\times$ slower
- SNN-Conv vs Transformer: $3.77/0.12 = 31.4\times$ slower

Training shows similar ratios: SNN-Conv takes $4663/154 = 30.3\times$ longer than CNN.

3.3 Parameter Efficiency

SNN-Conv uses 11,274 parameters for 97.41% accuracy. CNN uses 54,314 ($4.8\times$ more) for 97.76%. The spiking architecture achieves comparable accuracy with substantially fewer parameters, suggesting that temporal dynamics partially substitute for spatial filter capacity.

4 Discussion

4.1 Why SNNs Are Slow on CPUs

Each timestep requires a full forward pass: membrane update, threshold comparison, reset, and weight multiplication for all neurons. On a CPU, these operations cost the same whether a neuron fires or not. The spike sparsity (SNN-FC averages 991 spikes out of a theoretical maximum of $128 \times 25 = 3,200$) is invisible to the hardware because the membrane computation runs regardless.

On neuromorphic chips, only firing neurons trigger downstream computation. The 991 spikes would translate to 991 compute events instead of 3,200. This is where the theoretical $3\times$ to $10\times$ energy advantage materializes.

4.2 Practical Implications

For edge deployment on conventional processors (Jetson Nano, Raspberry Pi, microcontrollers), conventional CNN or MLP remains the practical choice. The accuracy parity of SNNs is encouraging for the neuromorphic hardware roadmap, but does not translate to a deployment advantage on existing silicon.

The parameter efficiency of SNN-Conv (11K vs 54K) is relevant for extremely memory-constrained targets (sub-1MB model budgets) where the model must fit in SRAM. In such cases, the SNN’s smaller footprint may justify the inference latency if the application can tolerate it.

4.3 Limitations

1. MNIST is a simple benchmark. More complex tasks (CIFAR-10, event camera data) may show different accuracy gaps.
2. Our S4/Mamba implementation in the prior study was simplified. A full Mamba comparison would be more informative.
3. We do not measure actual energy consumption, only proxy metrics (FLOPs, spike counts). True energy comparison requires hardware power measurement.
4. Rate coding is the simplest input encoding. Temporal coding or learned encodings may improve SNN efficiency.

5 Conclusion

Spiking neural networks achieve accuracy within 0.35 percentage points of conventional CNNs on MNIST classification, confirming that surrogate gradient training bridges the accuracy gap. However, on conventional CPU hardware, SNNs are $30\text{--}37\times$ slower at inference due to sequential timestep computation that cannot exploit spike sparsity. The neuromorphic advantage is real but hardware-dependent: SNNs are a software solution waiting for its hardware platform, analogous to deep learning before GPU acceleration.

Data and Code Availability

All code and results: <https://github.com/turingrtss/vulndetect>

References

- [1] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [2] M. Davies *et al.*, “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [3] P. A. Merolla *et al.*, “A million spiking-neuron integrated circuit with a scalable communication network and interface,” *Science*, vol. 345, no. 6197, pp. 668–673, 2014.
- [4] E. O. Neftci, H. Mostafa, and F. Zenke, “Surrogate gradient learning in spiking neural networks,” *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, 2019.